

CiT Backend Engineering Track

Week 5: Java Fundamentals — Part 3

Polymorphism, Abstraction, Collections, Exceptions & Modern Java

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 5
Instructor	Jim & Anderson
Duration	6 months 24 weeks
Tech Stack	Java 21, IntelliJ IDEA, Gradle

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we complete the foundation of the language we will build our careers upon. As we learn to model the world through abstraction, to organise data into ordered collections, and to handle the things that go wrong with grace, teach us to do all things decently and in order. Let the discipline we practise in our code reflect the discipline You call us to in life.

As 1 Corinthians 14:40 reminds us: "Let all things be done decently and in order." Lord, let that order shape how we structure our systems and our thinking.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

"Let all things be done decently and in order."

— 1 Corinthians 14:40 (NKJV)

Table of Contents

1. Welcome to Week 5
2. Polymorphism — The Third Pillar of OOP
3. Abstraction — Abstract Classes & Interfaces
4. The Collections Framework — List, Set & Map
5. Iterating Collections — Loops & Streams
6. Exception Handling
7. Modern Java 21 — Records, Sealed Classes & Pattern Matching
8. Project Kickoff — Student Grade Management System
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 5

Last week you crossed the most important threshold in the track: you moved from procedural code into Object-Oriented Programming. You can now define classes, create objects, protect their state with encapsulation, and build hierarchies with inheritance.

This week we complete the Java Fundamentals series. We finish the four pillars of OOP by adding polymorphism and abstraction, then we step beyond single objects into the Collections Framework — the tools Java gives you for managing groups of objects. We learn to handle things that go wrong with exception handling, and we taste the modern features that make Java 21 a pleasure to write.

Finally, this week your first substantial project begins: the Student Grade Management System. It will pull together everything from Weeks 3 to 5 into one real program. Treat this week as the bridge between learning the language and building with it.

What to Expect in Week 5

By the end of this session, you will be able to:

- Explain polymorphism and use method overriding to make one call behave in many ways
- Decide when to use an abstract class versus an interface, and implement both
- Use the core collections — ArrayList, HashMap, and HashSet — with generics
- Iterate collections with the enhanced for loop and the Stream API
- Handle errors safely with try / catch / finally and write your own custom exceptions
- Use modern Java 21 features: records, sealed classes, and pattern matching
- Begin building the Student Grade Management System project

2. Polymorphism — The Third Pillar of OOP

Last week you met two of the four pillars: encapsulation and inheritance. This week opens with the third — polymorphism. It is a long word, but the idea behind it is one you already understand from everyday life.

2.1 What Does "Polymorphism" Mean?

The word comes from Greek: "poly" means many, and "morph" means form. Polymorphism means "many forms". In programming, it means that a single action can behave differently depending on the object that performs it.

Think of the word "drive". You can drive a car, drive a motorbike, and drive a truck. The instruction is the same — "drive" — but what actually happens is different for each vehicle. You, the driver, do not need to know the internal details. You just say "drive", and each vehicle responds in its own way.

That is polymorphism: the same instruction (a method call) produces different behaviour depending on the actual object receiving it.

2.2 The Two Kinds of Polymorphism

Java gives you polymorphism in two forms. You have already used one of them.

Kind	Plain English Explanation
Compile-time (Overloading)	Several methods share the same name but take different parameters. The compiler decides which one to run based on the arguments you pass. You met this in Week 3 as method overloading.
Run-time (Overriding)	A child class provides its own version of a method it inherited from a parent. The JVM decides which version to run at run-time, based on the actual object — not the declared type. This is the powerful form we focus on today.

2.3 Method Overriding in Action

Imagine a payments system. Every payment can be processed, but a mobile money payment, a card payment, and a bank transfer each work differently. We model this with a parent class and three children that each override the same method.

```
public class Payment {
    protected double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    // The general behaviour — children will replace this
    public void process() {
        System.out.println("Processing a generic payment of UGX " + amount);
    }
}
```

```
public class MobileMoneyPayment extends Payment {
    public MobileMoneyPayment(double amount) { super(amount); }

    @Override
    public void process() {
        System.out.println("Sending UGX " + amount + " via Mobile Money...");
    }
}
```

```
public class CardPayment extends Payment {
    public CardPayment(double amount) { super(amount); }

    @Override
    public void process() {
        System.out.println("Charging UGX " + amount + " to a bank card...");
    }
}
```

2.4 Why This Is Powerful — One Type, Many Behaviours

The magic appears when we treat all these objects as their shared parent type. We can put different payment types in the same list and call the same method on each — and each responds in its own way.

```
Payment[] payments = {  
    new MobileMoneyPayment(50000),  
    new CardPayment(120000),  
    new Payment(1000)  
};  
  
for (Payment p : payments) {  
    p.process(); // each object runs ITS OWN version  
}
```

Output:

```
Sending UGX 50000.0 via Mobile Money...  
Charging UGX 120000.0 to a bank card...  
Processing a generic payment of UGX 1000.0
```

Why Engineers Love This

Notice that the for loop does not contain a single if statement checking the payment type. It does not care what kind of payment each object is. It simply says "process yourself", and each object knows how.

Tomorrow your team could add a CryptoPayment class. This loop would handle it correctly without a single change. That is the power of polymorphism: you write code against the general type, and new specific types slot in for free.

The @Override Annotation — Always Use It

@Override is not required by the compiler, but you should always write it above an overriding method. It tells the compiler "I intend to override a parent method". If you misspell the method name or get the parameters wrong, the compiler will catch your mistake instead of silently creating a brand-new method that never runs.

Exercise 1: Polymorphic Notifications

Create a parent class Notification with a method send(). Create three subclasses — EmailNotification, SmsNotification, and PushNotification — each overriding send() to print a different message.

- Store all three in a `Notification[]` array
- Loop through the array and call `send()` on each — confirm each prints its own message
- Add `@Override` on every overriding method

3. Abstraction — Abstract Classes & Interfaces

Abstraction is the fourth and final pillar of OOP. It is the practice of hiding complex details behind a simple, well-defined surface — showing what something does, not how it does it.

You use abstraction every time you drive a car. You press the accelerator and the car speeds up. You do not need to understand fuel injection, combustion, or the gearbox. The pedal is a simple interface to a very complex machine. Java gives you two tools to build this kind of simple surface: abstract classes and interfaces.

3.1 Abstract Classes

An abstract class is a class that cannot be instantiated on its own — you cannot write `new Shape()`. It exists only to be a parent. It can define some complete methods AND some abstract methods, which have no body and must be filled in by each child.

```
public abstract class Shape {
    private String name;

    public Shape(String name) { this.name = name; }

    // Abstract method — NO body. Every child MUST provide one.
    public abstract double area();

    // Concrete method — shared by all children
    public void describe() {
        System.out.println(name + " has an area of " + area());
    }
}
```

```
public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

```
}
```

3.2 Interfaces

An interface is a pure contract. It lists method signatures — names, parameters, and return types — but no implementation (with some modern exceptions). Any class that "signs the contract" promises to provide a body for every method listed.

Where a class uses extends to inherit from one parent, it uses implements to fulfil an interface — and it can implement many interfaces at once.

```
public interface Payable {  
    double calculatePay(); // a method with no body — just the signature  
}  
  
public class Employee implements Payable {  
    private double monthlySalary;  
  
    public Employee(double monthlySalary) {  
        this.monthlySalary = monthlySalary;  
    }  
  
    @Override  
    public double calculatePay() {  
        return monthlySalary;  
    }  
}
```

3.3 Abstract Class vs Interface — Which One?

Both let you define a method without implementing it. Beginners ask: when do I use which? The table below is your decision guide.

Use an Abstract Class when...	Use an Interface when...
Classes share common state (fields) and some shared behaviour	You only want to define a contract of capabilities, with no shared state
You are modelling a strong "is-a" relationship	You are modelling a "can-do" capability that unrelated classes might share

You need constructors and instance fields	You need a class to take on multiple capabilities at once
Example: a Shape that all shapes extend	Example: Payable, Comparable, Printable — capabilities any class can adopt

The Simple Rule of Thumb

A class can extend only ONE abstract class but can implement MANY interfaces. So ask: "Is this a thing it IS, or a thing it CAN DO?" An "is-a" usually points to an abstract class; a "can-do" usually points to an interface.

Exercise 2: Abstraction in Practice

Create an abstract class Employee with a field name, a concrete method clockIn(), and an abstract method calculateMonthlyPay().

- Create SalariedEmployee (fixed monthly salary) and HourlyEmployee (rate × hours) that each implement calculateMonthlyPay()
- Create an interface Taxable with a method calculateTax(), and have both employee types implement it
- Print each employee's pay and tax

4. The Collections Framework — List, Set & Map

So far, when you needed to store several values, you used an array. Arrays work, but they have a serious limitation: their size is fixed forever the moment you create them. If you make an array of 10 and need an 11th, you are stuck. Real programs rarely know in advance exactly how many items they will hold.

Java's answer is the Collections Framework — a set of ready-made, resizable, intelligent containers for groups of objects. The three you will use constantly are List, Set, and Map.

4.1 Generics — The Angle Brackets

Before the collections, one piece of syntax: the angle brackets, like `List<String>`. This is called generics. It tells the collection exactly what type of object it is allowed to hold.

Think of it as labelling a storage box "Books only". `List<String>` means "a list that holds only Strings". If you try to put an integer into it, the compiler stops you immediately — catching a bug before the program ever runs.

4.2 List & ArrayList — An Ordered, Resizable Sequence

A List is an ordered collection that allows duplicates and grows as needed. The most common implementation is `ArrayList`. Think of it as a shopping list — items have an order, you can add as many as you like, and the same item can appear twice.

```
import java.util.ArrayList;
import java.util.List;

List<String> students = new ArrayList<>();

students.add("Alice");    // add to the end
students.add("Bob");
students.add("Carol");

String first = students.get(0); // read by index → "Alice"
int count = students.size(); // how many → 3
students.remove("Bob");    // remove by value
boolean has = students.contains("Carol"); // → true
```

4.3 Set & HashSet — A Bag of Unique Items

A Set is a collection that does not allow duplicates. Adding the same value twice has no effect. Use it when uniqueness matters — for example, a set of national ID numbers, where each must appear once.

```
import java.util.HashSet;
import java.util.Set;

Set<String> emails = new HashSet<>();
emails.add("a@cit.org");
emails.add("b@cit.org");
emails.add("a@cit.org"); // ignored — already present
System.out.println(emails.size()); // 2, not 3
```

4.4 Map & HashMap — Key-Value Pairs

A Map stores pairs: a unique key and a value attached to it. Think of a dictionary — you look up a word (the key) to find its definition (the value). Or a class register — you look up a registration number (key) to find a student (value).

```
import java.util.HashMap;
import java.util.Map;

Map<String, Double> balances = new HashMap<>();

balances.put("Alice", 150000.0); // key → value
balances.put("Bob", 20000.0);

double aliceBalance = balances.get("Alice"); // 150000.0
boolean hasBob = balances.containsKey("Bob"); // true
balances.put("Alice", 175000.0); // same key overwrites the value
```

4.5 Which Collection Should I Use?

Collection	Use When You Need...	Real Example
List	An ordered sequence, duplicates allowed, access by position	A queue of transactions to process
Set	A group where every item must be unique, order does not matter	The set of unique phone numbers that voted

Map	To look up a value quickly using a unique key	Account number → account balance
-----	---	----------------------------------

Exercise 3: A Class Register

Create a `Map<String, Integer>` called `scores` that maps a student's name to their exam score.

- Add at least five students with their scores
- Look up and print one specific student's score
- Update one student's score using `put` with the same key
- Print how many students are in the map using `size()`

5. Iterating Collections — Loops & Streams

Storing data in a collection is only useful if you can walk through it. This section covers the two ways you will iterate: the enhanced for loop you already know, and the modern Stream API.

5.1 The Enhanced For Loop

The enhanced for loop (also called the "for-each" loop) reads each item in a collection, one at a time, without you managing an index. You met it with arrays in Week 3; it works identically on collections.

```
List<String> students = List.of("Alice", "Bob", "Carol");

for (String student : students) {
    System.out.println("Hello, " + student);
}
```

Read the colon as "in": "for each student in students". To loop over a Map, iterate its `entrySet()`:

```
for (Map.Entry<String, Double> entry : balances.entrySet()) {
    System.out.println(entry.getKey() + " has UGX " + entry.getValue());
}
```

5.2 The Stream API — Describe What, Not How

A stream lets you process a collection by describing the operations you want — filter these, transform those, collect the result — instead of writing the mechanics of a loop. It reads almost like a sentence and is the modern, professional style.

```
import java.util.List;
import java.util.stream.Collectors;

List<Integer> scores = List.of(45, 72, 88, 31, 95, 60);

// Keep only passing scores (>= 50), then collect them into a new list
List<Integer> passed = scores.stream()
    .filter(score -> score >= 50)
    .collect(Collectors.toList());
```

```
System.out.println(passed); // [72, 88, 95, 60]
```

The three operations you will use most:

Operation	What It Does
<code>filter(condition)</code>	Keeps only the items that match a condition. Like a sieve.
<code>map(transformation)</code>	Transforms every item into something else — e.g. turn each name into its length.
<code>forEach(action)</code>	Performs an action on every item — e.g. print each one.

```
// map example: turn names into their lengths
List<Integer> lengths = students.stream()
    .map(name -> name.length())
    .collect(Collectors.toList());
```

The Arrow → A First Look at Lambdas

The `score -> score >= 50` syntax is a lambda — a tiny, anonymous, throwaway function. Read it as "given a score, return whether it is at least 50". You will study lambdas in depth later; for now, recognise them as the compact way to tell a stream operation what to do for each item.

6. Exception Handling

Programs run in an imperfect world. Files go missing, networks drop, users type letters where numbers belong, and you occasionally divide by zero. An exception is Java's way of signalling "something went wrong". If you ignore it, the program crashes. Exception handling is how you respond gracefully instead.

6.1 What an Exception Looks Like

When something goes wrong, Java "throws" an exception object describing the problem. If nobody catches it, the program stops and prints a stack trace.

```
int[] numbers = {1, 2, 3};
System.out.println(numbers[5]); // there is no index 5!
```

```
// Program crashes with:
// Exception in thread "main"
// java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
```

6.2 try / catch / finally

You wrap risky code in a try block. If an exception occurs, control jumps to the matching catch block instead of crashing. The optional finally block always runs, whether or not an exception happened — perfect for cleanup.

```
try {
    int result = 100 / divisor;    // risky — divisor might be 0
    System.out.println("Result: " + result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero!");
} finally {
    System.out.println("Calculation attempt finished.");
}
```

Block	Plain English
try	Wraps the code that might fail. Java attempts to run it.
catch	Runs only if a matching exception was thrown. This is where you recover gracefully.
finally	Runs no matter what — success or failure. Use it to release resources, close files, etc.

6.3 Checked vs Unchecked Exceptions

Java has two families of exceptions. Knowing the difference explains why some code forces you to handle errors and some does not.

Type	Explanation
Checked	The compiler forces you to handle these (with try/catch) or declare them. They represent recoverable, expected problems — e.g. a file not being found (IOException).
Unchecked (Runtime)	The compiler does not force handling. They usually signal programming bugs — e.g. NullPointerException, ArrayIndexOutOfBoundsException. Fix the bug rather than catching it.

6.4 Throwing Your Own Exceptions

You can signal a problem yourself with the throw keyword, and you can define your own exception type by extending an existing one. This makes your error messages meaningful to your domain.

```
public class InsufficientFundsException extends RuntimeException {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

```
public void withdraw(double amount) {  
    if (amount > balance) {  
        throw new InsufficientFundsException(  
            "Cannot withdraw UGX " + amount + " — balance is only UGX " + balance);  
    }  
    balance = balance - amount;  
}
```

Exercise 4: Safe Division

Write a method `divide(int a, int b)` that returns `a / b`. Wrap the division in a try/catch so that dividing by zero prints a friendly message instead of crashing.

- Catch the `ArithmeticException` and print "Division by zero is not allowed."

- Add a finally block that prints "Done." every time
- Test it once with a valid divisor and once with zero

7. Modern Java 21 — Records, Sealed Classes & Pattern Matching

Java has evolved enormously. Java 21 (the version you installed in Week 1) includes features that make code shorter, safer, and clearer. You will not master them today, but you should recognise and start using them.

7.1 Records — Data Classes in One Line

Often you just need a simple class to hold data — a few fields, a constructor, getters, and equality. Writing all that by hand is tedious and error-prone. A record generates all of it for you from a single line.

```
// The old way — about 30 lines of fields, constructor, getters, toString...
```

```
// The record way — one line:
```

```
public record Student(String name, int age, double gpa) {}
```

```
// Use it like any class:
```

```
Student s = new Student("Alice", 22, 3.75);
```

```
System.out.println(s.name()); // "Alice" — accessor generated for you
```

```
System.out.println(s);        // Student[name=Alice, age=22, gpa=3.75]
```

A record automatically gives you the constructor, the accessors, a sensible `toString()`, and correct `equals()`/`hashCode()`. Use records for immutable data carriers.

7.2 Sealed Classes — Controlling Who Can Extend

Normally any class can extend your class. A sealed class lets you state exactly which classes are permitted to extend it, using the `permits` keyword. This keeps a hierarchy closed and predictable.

```
public sealed class Shape permits Circle, Rectangle {}
```

```
public final class Circle extends Shape {}
```

```
public final class Rectangle extends Shape {}
```

```
// Any other class trying to extend Shape will not compile.
```

7.3 Pattern Matching

Pattern matching removes the clumsy "check the type, then cast it" dance. The `instanceof` check can now declare a variable of the right type at the same time.

```
Object obj = "Hello CiT";

// Old way
if (obj instanceof String) {
    String text = (String) obj;    // manual cast
    System.out.println(text.length());
}

// Modern pattern matching — declares "text" automatically
if (obj instanceof String text) {
    System.out.println(text.length());
}
```

Why These Features Matter

Modern Java lets you say more with less code, and the compiler catches more mistakes for you. Records remove boilerplate, sealed classes make hierarchies safe and exhaustive, and pattern matching removes risky manual casts. Professional Java teams use these every day — start building the habit now.

8. Project Kickoff — Student Grade Management System

Everything you have learned across Weeks 3, 4, and 5 now comes together in your first substantial project: a command-line Student Grade Management System. You will build it over the coming sessions, and it will be the centrepiece of your Java fundamentals work.

8.1 What You Will Build

A program that runs in the terminal and lets a user manage students and their grades. It will read commands, store data in collections, calculate results, and handle invalid input gracefully — using OOP, collections, and exception handling together.

Core Features (Minimum)

- Add a new student with a name and registration number
- Record one or more subject scores for a student
- Calculate each student's average score and letter grade
- List all students with their averages
- Look up a single student by registration number
- Handle invalid input (e.g. a score above 100) without crashing

8.2 Suggested Design

Apply what you know. A sensible starting structure:

Class	Responsibility
Student	Holds a name, registration number, and a List of scores. Has methods to add a score and calculate its average and grade. A good candidate for a record or an encapsulated class.
GradeCalculator	Converts a numeric average into a letter grade (A/B/C/Fail). Pure logic, easy to test.
StudentService	Holds a Map of registration number → Student. Provides add, find, and list operations.
Main	Runs the menu loop, reads user input, calls the service, and handles exceptions.

8.3 This Week's Milestone

Do not try to build everything at once. By the end of this week, aim to complete the first milestone only:

1. Create the Student class (or record) with its fields
2. Create a List<Student> or Map<String, Student> to hold them
3. Add two or three students in code and print them all using a loop
4. Calculate and print one student's average score

How You Will Be Assessed

The project is graded on correctness, clean OOP design (encapsulation, sensible classes), appropriate use of collections, graceful error handling, and clear, well-named code committed to GitHub with meaningful messages. Build it incrementally and commit often.

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Polymorphism — one method call, many behaviours, decided at run-time by the actual object
- Method overriding with `@Override`, and why you should always annotate it
- Abstraction with abstract classes and interfaces, and how to choose between them
- The Collections Framework — List/ArrayList, Set/HashSet, Map/HashMap — with generics
- Iterating with the enhanced for loop and the Stream API (filter, map, forEach)
- Exception handling with try / catch / finally, checked vs unchecked, and custom exceptions
- Modern Java 21: records, sealed classes, and pattern matching
- How an OOP design, collections, and error handling combine in a real project

9.2 Homework

5. Complete all four in-class exercises if you did not finish them — push the code to your GitHub repository with clear commit messages
6. Build an AnimalSounds program: an abstract Animal class with an abstract makeSound(), and at least three subclasses that each override it. Store them in a list and loop through making each sound.
7. Complete this week's Student Grade Management System milestone — the Student class, a collection of students, and average calculation
8. Preview Week 6: Read about relational databases and what SQL is. Come with one question ready.

Coming Next — Week 6: Databases & SQL Foundations

Jim & Anderson will introduce how real systems store data permanently. You will learn relational databases, tables, and the SQL language for querying them — the bridge between your Java programs and the data they manage. The Student Grade Management System project continues alongside.

Closing Reflection

"Let all things be done decently and in order."

— 1 Corinthians 14:40 (NKJV)

This week you completed the Java foundation. Polymorphism, abstraction, collections, and clean error handling are the marks of an engineer who builds for the long term. Order in your code reflects order in your thinking. Keep practising — the project will turn this knowledge into skill.

Glossary of Terms

Term	Definition
Abstract Class	A class that cannot be instantiated and may contain abstract (bodyless) methods that subclasses must implement
Abstraction	Hiding complex implementation behind a simple interface — showing what something does, not how
ArrayList	A resizable, ordered List implementation — the most commonly used collection in Java
Collection	A resizable container for groups of objects (List, Set, Map) from java.util
Exception	An object representing an error or unexpected event that disrupts normal program flow
Generics	The angle-bracket syntax (e.g. List<String>) that fixes the type a collection may hold
HashMap	A Map implementation storing unique key-to-value pairs for fast lookup
Interface	A contract of method signatures that an implementing class must fulfil; a class can implement many
Lambda	A short anonymous function, e.g. <code>x -> x > 0</code> , used heavily with streams
Override	A subclass providing its own version of an inherited method; marked with <code>@Override</code>
Polymorphism	The ability of one method call to behave differently depending on the actual object
Record	A concise Java type for immutable data carriers, generating constructor, accessors and equals
Sealed Class	A class that restricts which other classes are permitted to extend it via "permits"
Stream	A pipeline for processing collections declaratively with operations like filter and map
try / catch	The construct used to attempt risky code and handle any exception it throws